



UNIVERSIDADE DE COIMBRA

Fundamentos de Inteligência Artificial
2025/2026 - 2º Semestre

Trabalho Prático N°1:
Lunar Lander
Uma abordagem reactiva

Trabalho realizado por:
Bernardo Direito, 2023215454,
uc2023215454@student.uc.pt;
Manel Carvalho, 2023212301,
uc2023212301@student.uc.pt;
Gonçalo Costa,
PL1

1. Introdução

O presente relatório documenta a análise, modelação e implementação de um agente autónomo capaz de controlar uma nave espacial no ambiente de simulação 2D *Lunar Lander*. O controlo da nave é efetuado através de um sistema de produções reativo, manipulando um motor principal e dois propulsores secundários (laterais), com o objetivo de contrariar a gravidade e a inércia, aterrando de forma segura numa plataforma delimitada entre duas bandeiras.

O projeto foi estruturado para responder a duas metas fundamentais: o desenvolvimento de um agente base para um ambiente estático (ausência de vento), e a subsequente evolução arquitetural para o Desafio Avançado, onde a nave enfrenta um ambiente dinâmico e turbulento (vento com força extrema de 20.0).

1.1. Manual de Utilização e Extensibilidade (Perspetivas)

Perspetiva do Utilizador: A execução do simulador foi desenhada para ser intuitiva através do ficheiro principal. No topo do ficheiro, o utilizador dispõe de constantes globais facilmente configuráveis para alterar as condições da simulação:

- **ENABLE_WIND** (Booleano): Ativa ou desativa a presença de vento lateral.
- **WIND_POWER** (Float): Define a força do vento (ex: 15.0 ou 20.0).
- **RENDER_MODE**: Pode ser definido como **'human'** para visualização gráfica do voo, ou **None** para testes de performance rápidos em *background*.
- **EPISODES** (Int): Define o número de episódios a simular, imprimindo no terminal a taxa de sucesso e a média de passos no final da execução.

Perspetiva do Programador: A arquitetura do código foi estruturada para garantir uma elevada modularidade e facilidade de adaptação. O sistema está dividido em três blocos distintos:

1. **Perceções:** Funções puras que recebem o vetor de observação e devolvem valores lógicos (**True/False**). Para adicionar um novo comportamento ou limite ao agente, basta criar uma nova função de perceção neste bloco.
2. **Ações:** Funções que encapsulam a mecânica da nave, devolvendo arrays NumPy estáticos (ex: **[1.0, 0.0]**).
3. **Agente Reativo:** A função **reactive_agent(observation)** atua como o motor de inferência. As regras estão encadeadas num bloco **if-elif**, o que significa que a prioridade de uma regra é definida puramente pela sua posição *top-down*. Adicionar ou alterar o comportamento do robô resume-se a inserir um novo **elif** na camada de prioridade correta.

2. Modelação do Agente Base (Cenário Sem Vento)

2.1. Arquitetura: Agente Reativo Simples

Para o cenário sem vento, o ambiente é totalmente observável em relação às variáveis necessárias para a navegação. Como tal, a arquitetura implementada foi a de um **Agente Reativo Simples (Simple Reflex Agent)**. O agente toma decisões baseando-se

exclusivamente no estado atual do ambiente, sem recurso a memória temporal ou planeamento a longo prazo.

2.2. Formalização do Espaço de Percepções e Ações

O vetor de observação do *Lunar Lander* é composto por 8 variáveis contínuas. Para que o agente pudesse operar através de um sistema de produções puramente lógico (IF-THEN), o espaço contínuo foi discretizado em **Percepções Booleanas**, traduzindo números reais em conceitos de alto-nível:

- **Emergência Lógica:** `perigo_capotar_esquerda` e `perigo_capotar_direita` avaliam se o ângulo (θ) e a velocidade angular (v_{θ}) ultrapassam os limites de segurança da nave (acionados a 0.25 radianos num cenário normal).
- **Navegação e Trajetória:** `a_fugir_esquerda` e `a_fugir_direita` combinam a posição horizontal (x) com a velocidade (v_x) para inferir desvios. O estado ideal é validado por `em_cima_da_plataforma_estavel`.
- **Controlo Vertical:** `queda_perigosa` ($v_y < -0.35$), `queda_rapida` ($v_y < -0.15$) e `quase_no_chao` ($y < 0.2$) inferem a proximidade ao solo e taxas de descida críticas.

As **Ações (Atuadores Base)** atuam de forma discreta selecionando mecânicas pré-definidas no formato [`motor_principal`, `motores_laterais`]:

- `desligar_motores()`: [0.0, 0.0] (Ação por defeito, queda livre controlada).
- `ligar_motor_principal()`: [1.0, 0.0] (Impulso frontal na direção do nariz da nave).
- `ligar_motor_esquerdo()`: [0.0, 1.0] (Aplica torque para a direita / rotação no sentido horário).
- `ligar_motor_direito()`: [0.0, -1.0] (Aplica torque para a esquerda / rotação anti-horária).

(Nota: A intensidade do motor principal é dinamicamente ajustada em certos blocos do controlo vertical para garantir o touchdown suave, mantendo contudo a natureza reativa no instante).

2.3. Sistema de Produções e Resolução de Conflitos

A tomada de decisão é governada por uma arquitetura hierárquica IF-ELIF. A ordem de avaliação atua nativamente como o mecanismo de resolução de conflitos, garantindo que as emergências se sobrepõem à navegação de rotina:

- **Atualização de Modelo Interno (Memória):** Estima e acumula o desvio provocado por fatores externos de forma constante.
- **R1 (Sobrevivência - Prioridade Máxima):** Prevenção imediata de capotamento. Ignora qualquer desvio de trajetória se a orientação da nave estiver em risco.
- **R2 (Alinhamento Fino):** Correção de pequenos desvios angulares se a nave já estiver estável (corredor central) sobre a plataforma.
- **R3 (Proatividade Anti-Vento):** Se o modelo de memória detetar turbulência (`sente_vento_forte`), força a nave a adotar uma inclinação de segurança extrema e dispara o motor principal contra a força externa.
- **R4 (Navegação/Fugas Normais):** Correção de trajetória padrão cruzando a inércia com a inclinação para forçar o retorno à zona das bandeiras.

- **R5 (Controlo Vertical - Sobreposição Absoluta):** Bloco independente no final da iteração que avalia a altitude e sobrepõe uma travagem de emergência a qualquer manobra lateral em curso.

2.4. Experimentação, Análise e Otimização (Ambiente Estático)

Durante a experimentação inicial (1000 episódios), notou-se a ocorrência de derrotas por *Timeouts* (limite de 1000 passos excedido). A análise demonstrou que o agente sofria de "síndrome de perfeccionismo": ao tentar corrigir desvios milimétricos perto do chão ($\text{abs}(x) > 0.15$), abortava a aterragem, originando um efeito *iô-iô* (subir e descer repetidamente).

Solução: Foi introduzida "folga" (*slack*) nas perceções. A perceção `quase_chao_fora_do_sitio` passou a tolerar desvios laterais até 0.20 (limite das bandeiras) e o início da travagem (`quase_no_chao`) foi reduzido de 0.3 para 0.2 de altitude.

Resultados do Cenário Base: O agente reativo atingiu uma sólida **taxa média de sucesso de 80%**, ultrapassando taxativamente a métrica mínima exigida (40%). A média de passos das aterragens bem-sucedidas fixou-se em aproximadamente **580 passos**. Este valor demonstra que a prioridade mecânica do agente é a segurança: ele não apressa a descida, preferindo uma aproximação tática e cautelosa, o que justifica a sua alta taxa de sobrevivência sem exceder o limite de *Timeout*.

3. Desafio Avançado (Cenário Dinâmico com Vento)

A introdução de vento transformou o simulador num ambiente parcialmente inobservável (a força constante de arrastamento não é reportada no vetor de estado). Perante uma força de vento extrema (`WIND_POWER = 20.0`), a taxa de sucesso do agente base decaiu abruptamente, obrigando a uma reestruturação lógica.

3.1. Evolução para Agente Reativo Baseado em Modelo (Adaptativo)

Para lidar com a inobservabilidade parcial sem recorrer a heurísticas de "batota" (como ler a constante global do simulador `ENABLE_WIND`), a arquitetura evoluiu para um **Agente Reativo Baseado em Modelo (*Model-based Reflex Agent*)**.

Foi introduzida uma memória de curto-prazo estritamente limitada a um passo de simulação (`prev_obs` e `prev_estimate`). Através do cálculo da diferença de velocidade horizontal (`drift`) a cada frame, o agente estima internamente a força de deslocação (`calc_wind_push`).

Cumprindo as restrições teóricas da disciplina, **esta estimativa contínua não é utilizada para controlo numérico direto**. O valor numérico é classificado de forma lógica pela nova perceção central `sente_turbulencia()`, que retorna *Verdadeiro* apenas se o módulo da estimativa ultrapassar 0.005.

Esta abordagem tornou o agente puramente **autoadaptativo**: ele deduz sozinho a presença da tempestade com os seus próprios sensores e, como consequência lógica, altera dinamicamente os limiares das suas restantes perceções para um "Modo de Combate", sem depender de variáveis externas.

3.2. Análise de Falhas e Solução de Conflitos

Para suportar o processo de otimização, foi desenvolvido um *Debugger de Falhas* acoplado ao ciclo da função `simulate`, permitindo classificar o motivo exato de cada derrota. A ferramenta revelou dois problemas críticos gerados pelo vento extremo:

A. O Efeito de Arrastamento ("Hovering")

- **Problema:** Sob vento a 20.0, a grande maioria das falhas eram "Saídas do Ecrã". Ao travar a descida aos `-0.35`, a nave pairava no ar, atuando como uma vela que era inevitavelmente arrastada para fora dos limites do cenário.
- **Solução (Estratégia de Queda Rápida/Controlada):** Quando a nave `sente_turbulencia()`, o limiar de `queda_perigosa` relaxa dinamicamente para `-0.55`. Esta estratégia de descida mais rápida minimiza o tempo de exposição da fuselagem à rajada lateral, adiando a travagem máxima para os últimos instantes da aterragem.

B. Conflito de Regras Hierárquico (*Rule Override*)

- **Problema:** Para contrariar o vento, a nave requer uma geometria de voo agressiva (inclinada quase a 30° para utilizar o vetor horizontal do motor principal). No entanto, a regra primordial de segurança (`perigo_capotar`, avaliada em primeiro lugar) estava a ativar aos 0.25 radianos, anulando o esforço lateral e forçando a nave a endireitar-se prematuramente. A nave lutava contra si própria.
- **Solução (Tolerância Dinâmica):** Quando o modelo interno deteta turbulência, a percepção de capotamento é relaxada para disparar apenas acima dos 0.6 radianos (~34°). Isto resolveu o conflito hierárquico no sistema de produções, dando margem à nave para lutar ativamente contra o ambiente. Exigiu-se ainda que a percepção `a_fugir` ficasse muito mais sensível (0.02 face a 0.08 sem vento) para garantir reflexos imediatos à força invisível.

3.3. Análise Experimental e Resultados (Desafio Avançado)

Sob uma força de vento extrema (`WIND_POWER = 20.0`), e após uma bateria de testes de 1000 episódios, o agente adaptativo registou uma **taxa de sucesso de 30%**. Considerando o impacto avassalador deste nível de turbulência mecânica, manter quase um terço das aterragens imaculadas com uma arquitetura puramente reativa é um resultado taticamente admirável.

A análise métrica mais reveladora deste cenário encontra-se na **média de passos das aterragens bem-sucedidas, que subiu de ~580 para cerca de 610 passos**. O contraste desta métrica face ao cenário estático comprova matematicamente o comportamento do sistema de produções: perante o perigo, a nave entra proativamente num estado de combate prolongado. O aumento do número de passos demonstra que a nave despende mais tempo a sustentar o voo e a executar uma manobra de inclinação extrema ("Caranguejo"), garantindo a estabilização lateral contra o vento antes de se comprometer com a aterragem final.

4. Conclusões

O desenvolvimento deste trabalho prático cumpriu com êxito todos os objetivos genéricos propostos, demonstrando a robustez da modelação de agentes autónomos através de descrições de comportamento de alto-nível. A correta abstração e discretização de observações contínuas em perceções lógicas puras permitiu implementar ações reativas que garantiram uns robustos 80% de sucesso num ambiente estático.

No que diz respeito ao Desafio Avançado, conclui-se que a inobservabilidade parcial gerada por forças externas (vento) não inviabiliza uma abordagem reativa, nem obriga ao abandono desta em prol de controladores numéricos contínuos. A introdução de uma memória elementar cumpriu o objetivo de formalizar o conceito de "estado", permitindo ao agente estimar a hostilidade do seu meio e adaptar os seus próprios instintos de forma isolada.

A análise conjugada da taxa de sucesso final (30%) e da média de passos (610) evidencia o desenvolvimento de um comportamento defensivo inteligente e prolongado. Em suma, a hierarquização rigorosa do sistema de regras **IF-THEN**, aliada à adaptação dinâmica de limiares para evitar conflitos lógicos, provou ser uma arquitetura exímia para ambientes caóticos, resolvendo problemas de navegação complexos em tempo real.